Code review · Mayuri's AI service

Academic AI Service: merge review

Folder reviewed: `academic-ai-service/` (33 files, about 2,300 lines, every file read) · Checked against `frontend/API_CONTRACT.md` v2.5 §9 and §18 · 27 Oct 2026 · Prepared for Rohan Belsare

Verdict: not ready to merge as it is. The design is sound and stage 2 (CO → PO/PSO) is solid and matches the contract. But six things must change first. A fresh install can't start the service, stage 1 (unit → CO) doesn't accept or return the contract's shapes, and AI failures come back as the caller's fault. Once those are fixed it can go into the repository as a new top-level folder.

6	8	18 / 18
must fix before merging	should fix soon after	pytest tests pass (stage 2 only)

How this was checked

- Every source, test, prompt and doc file was read line by line. The `.venv` folder was skipped, and only the variable names in `.env` were checked, never the values.
- A clean copy without `.venv` and `.env` was tested on Python 3.12, using a dummy API key so nothing called Gemini.
- A behaviour probe sent requests through FastAPI's test client with the LLM mocked: a contract-shaped stage 1 request, LLM output wrapped in Markdown, a hallucinated unit number, a Gemini failure, and a malformed request body.
- Not checked: real Gemini calls and answer quality. Those need the API key and the Network Security sample.

What's already good

- GOOD Stage 2's request (`CamSuggestionContext`) and response (`CamSuggestionSet`) match contract §9.3 and §9.5.
- GOOD Stage 2 validation is thorough. It checks unknown IDs, duplicate pairs, rubric levels, evidence units tied to the confirmed mapping, evidence items tied to the CO, and that non-blank levels need unit evidence. It also refuses before calling the LLM when a CO has no confirmed unit.
- GOOD The prompts carry Mayuri's rubric rules: evidence over similar wording, flagging doubtful cases, and never suggesting item links. They're versioned through template IDs.
- GOOD The LLM provider sits behind an interface (`BaseLLMProvider`), so switching models or providers is one class.
- GOOD Responses use the contract envelope (`success`, `data`, `errorCode`, `traceId`, `timestamp`).
- GOOD `.gitignore` already excludes `.venv`, `.env` and caches, so neither the virtual environment nor the key would be committed.
- GOOD `docs/stage-2-integration-notes.md` correctly states that Spring Boot must verify the confirmed mapping and CAM version before calling this service.

Must fix before merging

MUST 1 **A fresh install can't start the service**

`requirements.txt` is saved as UTF-16 (a PowerShell `>` redirect), and it's missing packages the code imports. A teammate who runs `pip install -r requirements.txt` gets a service that fails at start-up.

Missing package	Used by
`google-genai` (2.33.0 is in Mayuri's .venv)	`app/services/llm/providers/gemini.py:2`, imported on start-up
`httpx`	FastAPI's `TestClient` in the tests
`pytest`	the test suite

Change

- Regenerate it as UTF-8, for example `python -m pip freeze | Out-File -Encoding utf8 requirements.txt`, or keep a short hand-written list.
- Add `google-genai`. Put `pytest` and `httpx` in a `requirements-dev.txt`.

MUST 2 Stage 1 (unit → CO) doesn't accept the contract's request

Contract §9.5 defines one `CamSuggestionContext` for both stages. Stage 1 instead uses its own `UnitCoMappingContext` with a different shape, and it rejects unknown fields (`extra="forbid"`). So the request Spring Boot will send is refused.

Field	Contract §9.5 (what Spring sends)	Stage 1 today
Wrapper	`stage`, `offeringId`, `course`, `inputs`	Objectives, COs and units at the top level
Objectives	`string[]`	`{ objectiveId, statement }[]`
COs	`{ coId, code, statement }`	`{ coId, statement }`, no `code`
Units	`{ unitNumber, title, content, hours }`	`{ unitNumber, title, topics[], teachingHours }`
CAM version	`basedOnCamVersion` (as in stage 2)	Not accepted

Proof (probe 1)

```
contract-shaped stage 1 request -> 422  fieldErrors: context.courseOutcomes, context.units,
context.stage, context.offeringId
```

Change

- Let stage 1 take the same `CamSuggestionContext` and `CamInputs` as stage 2 (`app/schemas/cam_stage2_context.py`), with `stage: "UNIT_CO_MAPPING" | "CO_PO_CORRELATION"` and `basedOnCamVersion` on the request.
- Build the prompt from `inputs.units[].content` and `hours`, and include each CO's `code`.
- Files: `app/schemas/cam_context.py`, `app/schemas/cam_ai_request.py`, `app/pipelines/cam/unit_co_mapping.py`, `app/api/routes/cam.py:34-90`.

MUST 3 Stage 1's response doesn't match the contract

The contract says `UnitCoSuggestionSet extends AiProvenance`: the provenance fields sit flat on the set, and `basedOnCamVersion` is a number. Stage 2 already does this; stage 1 doesn't.

Proof (probe 2)

```
stage 1 response keys -> basedOnCamVersion, provenance, suggestionSetId, suggestions
basedOnCamVersion = None   (never set; typed str | int | None)
```

Change

- Make `UnitCoSuggestionSet` extend `AiProvenance` like `CamSuggestionSet` does, with flat `modelUsed`, `templateId`, `templateVersion` and `generatedAt`.
- Type `basedOnCamVersion` as `int` and fill it from the request.
- Files: `app/schemas/cam.py:62-76`, `app/api/routes/cam.py:77-84`.

MUST 4 AI failures are reported as the caller's mistake

When Gemini returns bad JSON or invents an ID, the pipelines raise `ValueError`, and the routes turn every `ValueError` into **400 VALIDATION_FAILED**. When Gemini errors out or times out, nothing catches it, so the service answers **500 in plain text** with no error envelope. Spring Boot can't tell its own bad request from an AI outage, so the faculty would see the wrong message.

Proof (probes 3-5)

```
LLM wraps JSON in ```json fences  -> 400 VALIDATION_FAILED "The LLM returned invalid JSON..."
LLM invents unit number 9         -> 400 VALIDATION_FAILED
Gemini raises an exception        -> 500 text/plain "Internal Server Error"
```

Change (contract §18)

- Use exception classes instead of `ValueError` everywhere:
  - `InvalidContextError` → 400 `VALIDATION_FAILED`;
  - `PreconditionError` → 422 `PRECONDITION_FAILED`;
  - `LlmOutputError` (bad JSON, unknown IDs) → 502 `AI_SERVICE_UNAVAILABLE`;
  - provider and network errors → 503 `AI_SERVICE_UNAVAILABLE`, or 504 `UPSTREAM_TIMEOUT` for timeouts.
- Replace the text match `"confirmed unit mapping" in message.lower()` in `app/api/routes/cam.py:113` with the `PreconditionError` type.
- Add a catch-all exception handler in `app/main.py` that returns the `ApiError` envelope with 500 `INTERNAL_ERROR` and a `traceId`, so no plain-text 500 escapes.

MUST 5 Malformed requests return 422 instead of 400

Contract §18 uses **400 VALIDATION_FAILED** for a bad request body and keeps 422 for `PRECONDITION_FAILED`. The request-validation handler in `app/main.py:20-43` returns 422.

```
malformed request body -> 422 VALIDATION_FAILED   (contract: 400 VALIDATION_FAILED)
```

Change

- Return `status_code=400` in `validation_exception_handler`, and update the three stage 2 tests that expect 422 for a missing or invalid `basedOnCamVersion`.

MUST 6 API key handling

- The shared folder's `.env` holds a real Gemini API key (a 39-character `AIza…` value; not reproduced here). `.gitignore` keeps it out of Git, but the key has already left Mayuri's machine with this folder. **Rotate it** in Google AI Studio, and share keys privately rather than inside project folders.
- Add the `.env.example` that `.gitignore` already allows (`GEMINI_API_KEY=`, `LLM_PROVIDER=gemini`, `LLM_MODEL=gemini-2.5-flash`), so teammates know which variables to set.
- The service can't even be imported without the key, because `Settings()` requires it at start-up (`app/core/config.py:19`). That's acceptable for running the service, but the README should say so, and tests should set a dummy key in a `conftest.py`.

Should fix soon after

#	Issue	Change
1	**Gemini call settings** (`providers/gemini.py:28-36`)	`temperature` is accepted but never passed on. Set `response_mime_type="application/json"`, which also prevents the Markdown fences from probe 3. Add a request timeout and one retry.
2	**One bad suggestion fails the whole set**	A single invented ID throws away every valid suggestion. Retry the LLM once with the validation error. If it still fails, return 502 (MUST 4) or drop the invalid items and log them.
3	**Stage 1 tests never run under pytest**	`test_cam_context`, `test_unit_co_prompt`, `_duplicates`, `_invalid_json` and `_validation` are scripts with `main()`, so pytest collects 0 tests from them. They pass when run by hand. Convert them to `test_*` functions, as `test_co_po_correlation.py` does.
4	**Live-API scripts sit in `tests/`**	`test_gemini_api.py` (hard-codes model `gemini-3.5-flash-lite`), `test_llm_connection.py` and `test_unit_co_generation.py` call Gemini for real. Move them to `scripts/`, or mark them `@pytest.mark.integration` and skip them when no key is set.
5	**Plain `pytest` fails**	Running `pytest` fails with `ModuleNotFoundError: app` in all 11 files; only `python -m pytest` works. Add `pyproject.toml` with `[tool.pytest.ini_options] pythonpath = ["."]`, plus a `conftest.py` that sets the dummy key.
6	**Service-to-service security**	The endpoints have no authentication, so anyone who can reach the port can spend the Gemini quota. Add a shared-secret header (for example `X-Internal-Token`) that Spring Boot sends, keep the port internal, and settle contract open question O-6.
7	**No README**	Document setup, the `.env` variables, the run command (`uvicorn app.main:app --port 8000`; Spring uses 8081), the test command, and the two internal routes Spring calls.
8	**Leftovers and tidy-up**	• Remove the empty `app/pipelines/cam/unit_mapping.py` and the empty `question_paper_validation/` folders. • Remove the unused `UnitCoMappingRequest` and `CamSuggestionRequest` (`schemas/cam.py:16-28, 78-90`) and the duplicate `current_timestamp`. • Add `__init__.py` to the `api`, `core`, `schemas`, `pipelines` and `prompts` packages. • Use one template-ID style: today it's `CAM_UNIT_CO_MAPPING` next to `cam_co_po_correlation`.

Test results

Check	Result

`pytest tests` (the plain command)	11 collection errors (`No module named 'app'`)
`python -m pytest tests` (project root on the path, dummy key)	**18 passed**: 13 stage 2 pipeline, 4 stage 2 route, 1 error envelope
Stage 1 scripts run by hand (mocked LLM)	5 of 5 pass: context, prompt, duplicates, invalid JSON, field validation
Live Gemini scripts	Not run: they need the real key and spend quota
Contract probes 1-6	Fail as described in MUST 2-5
Import without `GEMINI_API_KEY`	Fails at start-up (MUST 6)

Environment: Python 3.12, fastapi 0.116, pydantic 2.11, google-genai 1.49. These are slightly older than the pinned versions, because the pinned set couldn't be downloaded here.

How to merge once fixed

1. Copy the folder to the repository root, next to `Abhijeet/` and `frontend/`, for example as `Mayuri/academic-ai-service/`. Leave out `.venv/`, `.env` and every `__pycache__/`; its own `.gitignore` keeps them out.
2. Before committing, run `git status` and confirm no `.env` or `.venv` file is staged.
3. Install with the fixed `requirements.txt` in a fresh virtual environment, run `pytest` (all tests, including the converted stage 1 ones), and start the service once with a real key.
4. Abhijeet then calls `POST /cam/unit-mapping/suggestions` and `POST /cam/co-po-correlation/suggestions` from Spring Boot's CAM-5 and CAM-4, as written in contract §9.5 and Mayuri's integration notes.

Generated from a full read of the folder and test runs on a clean copy. File and line references point to the folder as received on 27 Oct 2026.